

Khalil Ahmad & Sellou Rayan - M1 ICE-LD

Projet de développement orienté données

1. La mise en place de l'entrepôt de données	3
1.1. L'outil de stockage choisi	3
1.2. Sources et qualité des données	3
1.3. Mise en place de l'ETL	6
1.4. Les données mises en place pour compléter	6
2. Besoins des gestionnaires de vols	7
2.1. Requêtes SQL	7
2.2. Résultats obtenus	9
3. Bilan	12
3.1. Répartition des tâches	12
Annexes	13
Script SQL de création de la base de données	13
Script Python d'insertion des données	17

Lien du répertoire github contenant tous les fichiers du projet :

<https://github.com/Marshall331/projet-entrepot-donnees>

1. La mise en place de l'entrepôt de données

1.1. L'outil de stockage choisi

Pour ce projet, notre choix s'est porté sur PostgreSQL comme système de gestion de base de données relationnelle. Ce choix s'appuie tout d'abord sur la fiabilité du moteur, qui respecte strictement les propriétés ACID.

Dans le cadre de la construction d'un entrepôt de données nécessitant l'ingestion de volumes importants via des processus ETL, cette garantie d'intégrité lors des transactions est primordiale. De plus, PostgreSQL est particulièrement réputé pour ses performances lors de l'exécution de requêtes analytiques et d'agrégations complexes, ce qui correspond parfaitement à nos besoins de reporting.

Nous avons également tiré parti de la richesse de ses contraintes intégrées, telles que les expressions régulières pour valider les formats de licences ou les vérifications strictes sur les valeurs numériques, permettant de bloquer les anomalies dès l'insertion. Enfin, sa stricte conformité au standard SQL assurera une interopérabilité sans faille avec des outils de Business Intelligence comme Power BI lors de la seconde phase du projet.

1.2. Sources et qualité des données

La pertinence et la justesse de nos analyses reposant entièrement sur la qualité des données initiales, nous avons opté pour une approche hybride garantissant à la fois le réalisme du contexte métier et la précision requise par nos scénarios de test.

Dans un premier temps, nous nous sommes appuyés sur les référentiels ouverts [d'OpenFlights](#) pour extraire une base réaliste concernant la structure du réseau aérien : aéroports principaux, codes IATA officiels (comme CDG pour Paris Charles de Gaulle ou JFK pour New York), et identifiants réels de compagnies aériennes.

Le tableau ci-dessous détaille la correspondance entre les fichiers sources bruts issus d'OpenFlights et les attributs intégrés dans notre modèle de données :

Le tableau ci-dessous détaille la correspondance entre les fichiers sources bruts issus d'OpenFlights et les attributs intégrés dans notre modèle de données :

Fichier source (OpenFlights)	Données extraites	Tables cibles	Données générées manuellement
countries.csv	Code ISO (2 lettres), Nom du pays	Pays	PopulationP (estimations démographiques réalistes)
airports.csv	Code IATA (3 lettres), Nom de l'aéroport, Ville	Airports, Villes	TailleHub, PopulationVi (tailles de hub et populations simulées)
airlines.csv	Code IATA (2 lettres), Nom de la compagnie	Compagnies	Affectation du siège social (CodeP) et du hub principal (CodeAP)
planes.csv	Nom du modèle, Constructeur	Avions	CodeA, NbrSieges, ChargeMax, PoidsAVide, AnneeAchat, affectation du type et de la compagnie
routes.csv	Aéroport de départ, Aéroport d'arrivée, Compagnie	VolsReferences	CodeVR, Frequence, Distance, HeureDepart, HeureArrivee, catégorie du vol

Pour combler les lacunes de ces jeux de données ouverts, plusieurs attributs ont dû être générés à la main ou simulés de manière stratégique :

- données référentielles complètes : les tables Categories et Types ont été entièrement créées sur mesure pour correspondre aux besoins de classification du projet. Les indicateurs de capacité et de démographie (TailleHub , PopulationVi , PopulationP) ont été ajoutés avec des valeurs réalistes pour enrichir les futurs axes d'analyse du magasin de données.
- données techniques et contraintes : les caractéristiques physiques des appareils (ChargeMax , PoidsAVide , NbrSieges) ont été estimées de façon cohérente selon le modèle d'avion. Cela était indispensable pour respecter la règle métier interdisant que le poids à vide dépasse la charge maximale. Le code d'immatriculation (CodeA) a quant à lui été généré spécifiquement pour valider la contrainte de format stricte (exactement deux lettres suivies de quatre chiffres).
- données transactionnelles (Table Vols) : les données des vols effectifs ont été créées de toutes pièces pour servir nos tests. Les dates (DateV), la durée du vol , la répartition des passagers et les charges (kérosène et charge totale) ont été minutieusement calibrées. Cette étape manuelle était cruciale pour simuler des scénarios spécifiques, comme forcer certains appareils de la compagnie Air France à atteindre le seuil de 5 vols sur la période de janvier 2025.

Cependant, ces bases publiques étant par nature incomplètes concernant les données d'exploitation pures (telles que le nombre précis de passagers par classe de réservation, les charges exactes de kérosène ou de fret), nous avons pris l'initiative de concevoir un jeu de données sur-mesure (à la main).

La qualité globale de ces données est par ailleurs garantie structurellement par le SGBD : avant même l'insertion effective, notre schéma relationnel filtre rigoureusement les informations en rejetant automatiquement les incohérences physiques métiers, comme un poids

à vide qui serait supérieur à la charge maximale autorisée pour un avion donné.

1.3. Mise en place de l'ETL

Concernant l'alimentation effective de l'entrepôt, nous avons fait le choix de développer une procédure d'ETL automatisée à l'aide d'un script Python. Pour interagir avec PostgreSQL, nous avons utilisé la bibliothèque `psycopg2`, reconnue pour sa robustesse. La complexité principale de notre schéma de données résidant dans ses nombreuses dépendances (clés étrangères), le script a été conçu pour respecter scrupuleusement un ordonnancement topologique strict lors de la phase de transformation.

Ainsi, l'insertion s'effectue de manière hiérarchique : le système ingère d'abord les tables de référence indépendantes dites de "niveau 0" (Pays, Catégories, Types, Aéroports), avant de passer au niveau 1 (Villes, Compagnies), puis au niveau 2 (Avions, Vols de référence), pour finalement terminer par la table de faits transactionnelle contenant les vols effectifs.

Afin d'optimiser la phase de chargement (Load), nous avons délibérément écarté l'usage de requêtes `INSERT INTO` classiques parcourant les fichiers ligne par ligne, car ces dernières s'avèrent beaucoup trop lentes et gourmandes en ressources pour des traitements par lots. À la place, notre script exploite la commande native PostgreSQL `COPY FROM STDIN`. Cette méthode lit directement les fichiers CSV et les transfère en mémoire de manière extrêmement rapide, constituant ainsi la norme industrielle pour ce type d'opération d'ingestion massive. Enfin, le script intègre une gestion sécurisée des transactions.

Chaque fichier CSV est traité de manière totalement isolée. Si l'importation se déroule sans erreur, la transaction est validée de manière permanente via une instruction `COMMIT`. En revanche, si une ligne du fichier viole une contrainte d'intégrité ou présente un défaut de formatage, le script intercepte l'exception et déclenche un `ROLLBACK`. Ce mécanisme annule l'intégralité de l'insertion en cours pour la table concernée, empêchant ainsi toute corruption de l'entrepôt avec des données partielles ou bancales.

1.4. Les données mises en place pour compléter

Conformément aux directives du cahier des charges, qui préconisaient de ne pas surcharger inutilement la base de données, nous avons construit nos jeux de données additionnels de manière hautement stratégique. L'objectif était d'insérer le volume minimum requis pour déclencher et valider chacun des scénarios analytiques demandés par les utilisateurs.

Nous avons ainsi créé des profils d'exploitation spécifiques pour certaines compagnies afin de servir de cas de test (vrais ou faux). Par exemple, pour valider la requête devant isoler les transporteurs n'opérant que sur des rythmes quotidiens, nous avons configuré les données de la compagnie Delta Airlines de sorte qu'elle ne possède exclusivement que ce type de vol en référence, tandis que ses concurrentes se voient attribuer des fréquences volontairement mixtes.

De la même manière, pour tester les requêtes à forte composante temporelle et conditionnelle, nous avons artificiellement concentré l'activité de certains appareils sur le premier trimestre de l'année 2025. C'est notamment le cas pour l'Airbus A320 d'Air France (immatriculé AF1001), auquel nous avons manuellement assigné plus de cinq vols distincts sur le seul mois de janvier. Cette préparation minutieuse en amont nous garantit que nos fonctions d'agrégations conditionnelles (comme la clause `HAVING COUNT`) puissent remonter les bons enregistrements et exclure les autres lors des tests de recette de notre base de données.

2. Besoins des gestionnaires de vols

Cette section présente les réponses techniques aux demandes d'analyse formulées par le consortium des compagnies aériennes.

2.1. Requêtes SQL

Requête (a) : nombre total de vols opérés par chaque compagnie.

Objectif : fournir le code, le nom de la compagnie et le pays du siège social, avec le volume de vols, trié par pays.

`SELECT`

`c.CodeC,`

`c.NomC,`

```

    p.NomP AS Pays_Siege_Social,
    COUNT(v.DateV) AS Total_Vols
FROM Compagnies c
JOIN Pays p ON c.CodeP = p.CodeP
LEFT JOIN VolsReferences vr ON c.CodeC = vr.CodeC
LEFT JOIN Vols v ON vr.CodeVR = v.CodeVR
GROUP BY c.CodeC, c.NomC, p.NomP
ORDER BY p.NomP ASC;

```

Requête (b) : compagnies opérant exclusivement des vols quotidiens.

Objectif : identifier les compagnies dont la stratégie repose uniquement sur une fréquence quotidienne. Nous utilisons une exclusion pour éliminer celles qui ont d'autres types de fréquences.

```

SELECT
    CodeC,
    NomC
FROM Compagnies
WHERE CodeC IN (SELECT CodeC FROM VolsReferences WHERE
    Frequence = 'Quotidien')
    AND CodeC NOT IN (SELECT CodeC FROM VolsReferences WHERE
    Frequence != 'Quotidien');

```

Requête (c) : Analyse de l'activité d'Air France en Janvier 2025.

Objectif : Lister les avions (code et modèle) ayant effectué au moins 5 vols durant ce mois spécifique.

```

SELECT

```



```
a.CodeA,  
a.Modele,  
COUNT(*) AS Nb_Vols_Janvier  
FROM Vols v  
JOIN Avions a ON v.CodeA = a.CodeA  
JOIN Compagnies c ON a.CodeC = c.CodeC  
WHERE c.NomC = 'Air France'  
AND v.DateV BETWEEN '2025-01-01' AND '2025-01-31'  
GROUP BY a.CodeA, a.Modele  
HAVING COUNT(*) >= 5;
```

2.2. Résultats obtenus

Sur la base de notre jeu de données de test, les résultats validés sont les suivants :

Résultat (a) : le système affiche une liste correctement triée par ordre alphabétique du pays du siège social. Par exemple, l'Allemagne (Lufthansa) et le Canada (Air Canada) apparaissent logiquement en tête de liste, avant l'Espagne (Iberia) ou la France (Air France). Suite à notre scénario de test d'insertion, on observe qu'Air France comptabilise 19 vols opérés, tandis que Delta Airlines en totalise 5. On remarque également la présence d'Air China avec 0 vol.

```

1  SELECT
2      c.CodeC,
3      c.NomC,
4      p.NomP AS Pays_Siege_Social,
5      COUNT(v.DateV) AS Total_Vols
6  FROM  Compagnies c
7  JOIN  Pays p ON c.CodeP = p.CodeP
8  LEFT JOIN VolsReferences vr ON c.CodeC = vr.CodeC
9  LEFT JOIN Vols v ON vr.CodeVR = v.CodeVR
10 GROUP BY c.CodeC, c.NomC, p.NomP
11 ORDER BY p.NomP ASC;

```

Data Output Messages Notifications

	codec character	nomc character varying (50)	pays_siege_social character varying (50)	total_vols bigint
1	LH	Lufthansa	Allemagne	3
2	AC	Air Canada	Canada	1
3	CA	Air China	Chine	0
4	IB	Iberia	Espagne	2
5	DL	Delta Airlines	Etats-Unis	5
6	AF	Air France	France	19
7	JL	Japan Airlines	Japon	2
8	BA	British Airways	Royaume-Uni	2

Résultat (b) : les compagnies Delta Airlines (DL) et Japan Airlines (JL) sont les seules retournées par la requête. Le filtre d'exclusion a parfaitement fonctionné : les compagnies comme Air France ou Iberia ont été écartées du résultat car, bien qu'elles possèdent des vols quotidiens, elles opèrent également d'autres fréquences (comme des vols hebdomadaires ou mensuels) dans notre base de référence.

Requête		Historique
1	SELECT	
2	CodeC,	
3	NomC	
4	FROM Compagnies	
5	WHERE CodeC IN (SELECT CodeC FROM VolsReferences WHERE Frequence = 'Quotidien')	
6	AND CodeC NOT IN (SELECT CodeC FROM VolsReferences WHERE Frequence != 'Quotidien');	

Data Output		Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈			
	codec		nomc
	[PK] character		character varying (50)
1	DL		Delta Airlines
2	JL		Japan Airlines

Résultat (c) : l'analyse ciblée de la flotte d'Air France identifie deux appareils répondant aux critères d'hyper-activité sur le mois de janvier 2025. L'avion immatriculé AF1001 (Modèle A320) apparaît en tête avec 7 vols, suivi de l'avion AF1003 (Modèle B777) avec 6 vols. Les autres appareils de la flotte (comme le A350 immatriculé AF1002) n'apparaissent pas car ils ne remplissent pas la condition stricte `HAVING COUNT(*) >= 5`.

Requête

Historique

1

2

3

4

5

6

7

8

9

10

11

SELECT

a.CodeA,

a.Modele,

COUNT(*) AS Nb_Vols_Janvier

FROM Vols v

JOIN Avions a ON v.CodeA = a.CodeA

JOIN Compagnies c ON a.CodeC = c.CodeC

WHERE c.NomC = 'Air France'

AND v.DateV BETWEEN '2025-01-01' AND '2025-01-31'

GROUP BY a.CodeA, a.Modele

HAVING COUNT(*) >= 5;

Data Output

Messages

Notifications

	codea [PK] character	modele character varying (8)	nb_vols_janvier bigint
1	AF1001	A320	7
2	AF1003	B777	6

3. Bilan

3.1. Répartition des tâches

Ce projet a été réalisé en binôme avec une organisation assez simple mais efficace. Au début, on s'est réparti certaines tâches pour avancer plus vite : l'un s'est occupé de la recherche des données et de la création des fichiers CSV (Ahmad), tandis que l'autre s'est concentré sur la création de la base de données sous PostgreSQL (Rayan).

Même si cette répartition nous a permis de bien démarrer, on a ensuite travaillé ensemble sur les parties les plus importantes. Notamment, on a conçu à deux le script Python pour l'insertion des données (avec COPY), en testant au fur et à mesure pour s'assurer que tout fonctionnait correctement.

On a aussi réalisé ensemble les requêtes de test pour vérifier les données et s'assurer que tout était cohérent dans la base.

Enfin, toute la partie rédaction du compte rendu a été faite ensemble. On a aussi pris le temps de relire et d'ajuster ce qu'on avait fait chacun, ce qui nous a permis d'avoir un rendu plus cohérent et de bien comprendre l'ensemble du projet.

Annexes

Script SQL de création de la base de données

```
-- 1. Nettoyage de la base de données
```

```
DROP TABLE IF EXISTS Vols CASCADE;
```

```
DROP TABLE IF EXISTS VolsReferences CASCADE;
```

```
DROP TABLE IF EXISTS Avions CASCADE;
```

```
DROP TABLE IF EXISTS Compagnies CASCADE;
```

```
DROP TABLE IF EXISTS Villes CASCADE;
```

```
DROP TABLE IF EXISTS Airports CASCADE;
```

```
DROP TABLE IF EXISTS Pays CASCADE;
```

```
DROP TABLE IF EXISTS Categories CASCADE;
```

```
DROP TABLE IF EXISTS Types CASCADE;
```

```
--
```

```
=====
```

```
==
```

```
-- 2. Création des tables de niveau 0
```

```
--
```

```
=====
```

```
==
```

```
CREATE TABLE Pays (
```

```
    CodeP CHAR(2) PRIMARY KEY,
```

```
    NomP VARCHAR(50) NOT NULL,
```

```
PopulationP BIGINT CHECK (PopulationP > 0)

);
```

```
CREATE TABLE Categories (

    CodeCa SERIAL PRIMARY KEY,

    NomCa VARCHAR(30) NOT NULL

);
```

```
CREATE TABLE Types (

    CodeTy SERIAL PRIMARY KEY,

    NomTy VARCHAR(30) NOT NULL

);
```

```
CREATE TABLE Airports (

    CodeAP CHAR(3) PRIMARY KEY,

    NomAP VARCHAR(50) NOT NULL,

    TailleHub INTEGER CHECK (TailleHub > 0)

);
```

```
--
=====
==
```

```
-- 3. Création des tables avec clés étrangères
```

```
--
=====
==
```

```
CREATE TABLE Villes (

    CodeVi VARCHAR(5) PRIMARY KEY,
```

```
NomVi VARCHAR(50) NOT NULL,  
PopulationVi BIGINT CHECK (PopulationVi > 0),  
CodeP CHAR(2) NOT NULL,  
FOREIGN KEY (CodeP) REFERENCES Pays(CodeP)  
);
```

```
CREATE TABLE Compagnies (  
    CodeC CHAR(2) PRIMARY KEY,  
    NomC VARCHAR(50) NOT NULL,  
    CodeP CHAR(2) NOT NULL,  
    CodeAP CHAR(3) NOT NULL,  
    FOREIGN KEY (CodeP) REFERENCES Pays(CodeP),  
    FOREIGN KEY (CodeAP) REFERENCES Airports(CodeAP)  
);
```

```
CREATE TABLE Avions (  
    CodeA CHAR(6) PRIMARY KEY CHECK (CodeA ~ '^[A-Za-z]{2}[0-9]{4}$'),  
    Modele VARCHAR(8) NOT NULL,  
    Constructeur VARCHAR(50) NOT NULL,  
    NbrSieges INTEGER NOT NULL CHECK (NbrSieges >= 0),  
    ChargeMax REAL NOT NULL CHECK (ChargeMax > 0),  
    PoidsAVide REAL NOT NULL CHECK (PoidsAVide > 0),  
    AnneeAchat INTEGER,  
    CodeTy INTEGER NOT NULL,  
    CodeC CHAR(2) NOT NULL,  
    FOREIGN KEY (CodeTy) REFERENCES Types(CodeTy),  
    FOREIGN KEY (CodeC) REFERENCES Compagnies(CodeC),
```

```

        CHECK (PoidsAVide < ChargeMax)

);

CREATE TABLE VolsReferences (

    CodeVR VARCHAR(8) PRIMARY KEY,

    Frequence VARCHAR(20) NOT NULL,

    Distance INTEGER NOT NULL CHECK (Distance > 0),

    HeureDepart TIME NOT NULL,

    HeureArrivee TIME NOT NULL,

    CodeAP_Dep CHAR(3) NOT NULL,

    CodeAP_Arr CHAR(3) NOT NULL,

    CodeCa INTEGER NOT NULL,

    CodeC CHAR(2) NOT NULL,

    FOREIGN KEY (CodeAP_Dep) REFERENCES Airports(CodeAP),

    FOREIGN KEY (CodeAP_Arr) REFERENCES Airports(CodeAP),

    FOREIGN KEY (CodeCa) REFERENCES Categories(CodeCa),

    FOREIGN KEY (CodeC) REFERENCES Compagnies(CodeC),

    CHECK (Frequence IN ('Quotidien', 'Quotidien semaine', 'Quotidien week end',
    'Hebdomadaire', 'Mensuel', 'Spécifique'))

);

```

```

CREATE TABLE Vols (

    CodeA CHAR(6) NOT NULL,

    CodeVR VARCHAR(8) NOT NULL,

    DateV DATE NOT NULL,

    DureeVol REAL NOT NULL CHECK (DureeVol >= 0),

    NbrPassagersEco INTEGER NOT NULL DEFAULT 0 CHECK (NbrPassagersEco >= 0),

```



```

    NbrPassagersAffaire INTEGER NOT NULL DEFAULT 0 CHECK (NbrPassagersAffaire >=
0),

    NbrPassagersPremiere INTEGER NOT NULL DEFAULT 0 CHECK
(NbrPassagersPremiere >= 0),

    ChargeKerosene REAL NOT NULL CHECK (ChargeKerosene > 0),

    ChargeTotale REAL NOT NULL CHECK (ChargeTotale > 0),

    PRIMARY KEY (CodeA, CodeVR, DateV),

    FOREIGN KEY (CodeA) REFERENCES Avions(CodeA),

    FOREIGN KEY (CodeVR) REFERENCES VolsReferences(CodeVR)

);

```

Script Python d'insertion des données

```

import psycopg2

import os

# 1. Configuration de la connexion à la base de données

DB_CONFIG = {

    "dbname": "aviation_db",

    "user": "admin",

    "password": "password123",

    "host": "localhost",

    "port": "5432"

}

# 2. Ordre d'insertion pour respecter les clés étrangères

# Format : ("Nom_de_la_table", "Nom_du_fichier.csv")

TABLES_ORDER = [

    # Niveau 0

```

```
("Pays", "csvs/Pays.csv"),  
("Categories", "csvs/Categories.csv"),  
("Types", "csvs/Types.csv"),  
("Airports", "csvs/Airports.csv"),
```

Niveau 1

```
("Villes", "csvs/Villes.csv"),  
("Compagnies", "csvs/Compagnies.csv"),
```

Niveau 2

```
("Avions", "csvs/Avions.csv"),  
("VolsReferences", "csvs/VolsReferences.csv"),
```

Niveau 3

```
("Vols", "csvs/Vols.csv")
```

```
]
```

```
def import_csv_to_postgres():
```

```
    conn = None
```

```
    try:
```

```
        # Connexion à la base de données
```

```
        print("Connexion à la base de données PostgreSQL...")
```

```
        conn = psycopg2.connect(**DB_CONFIG)
```

```
        cur = conn.cursor()
```

```
        for table_name, csv_file in TABLES_ORDER:
```

```
            # Vérification de l'existence du fichier
```

```

        if not os.path.exists(csv_file):

            print(f"ERREUR : Le fichier {csv_file} est introuvable. Importation annulée pour
cette table.")

            continue

    print(f"Importation en cours pour la table '{table_name}' depuis {csv_file}...")

    # Ouverture du fichier CSV et exécution de la commande COPY

    with open(csv_file, 'r', encoding='utf-8') as f:

        # COPY pour insérer des CSV

        # HEADER précise que la première ligne contient le nom des colonnes

        sql_copy_query = f"COPY {table_name} FROM STDIN WITH CSV HEADER
DELIMITER ','"

        try:

            cur.copy_expert(sql_copy_query, f)

            conn.commit() # On valide la transaction pour cette table

            print(f"Succès : Données insérées dans '{table_name}'.")

        except Exception as table_error:

            conn.rollback() # En cas d'erreur, on annule l'insertion pour cette table

            print(f"ERREUR SQL lors de l'insertion dans '{table_name}': {table_error}")

    # Fermeture du curseur

    cur.close()

    print("\nProcessus d'importation terminé !")

except psycopg2.OperationalError as e:

    print(f"\nErreur de connexion à la base de données : \n{e}")

```

```
except Exception as e:
```

```
    print(f"\nErreur inattendue : \n{e}")
```

```
finally:
```

```
    # Fermeture propre de la connexion quoi qu'il arrive
```

```
    if conn is not None:
```

```
        conn.close()
```

```
    print("Déconnexion de la base de données.")
```

```
if __name__ == "__main__":
```

```
    import_csv_to_postgres()
```